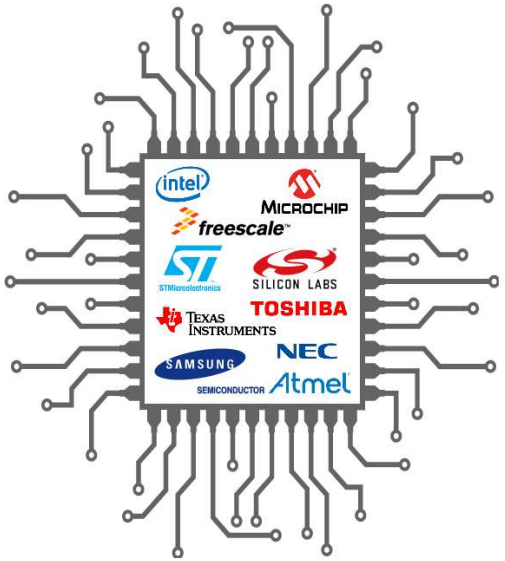
Analog to Digital Conversion



Thareendhra Keerthi Wijayasiriwardhane

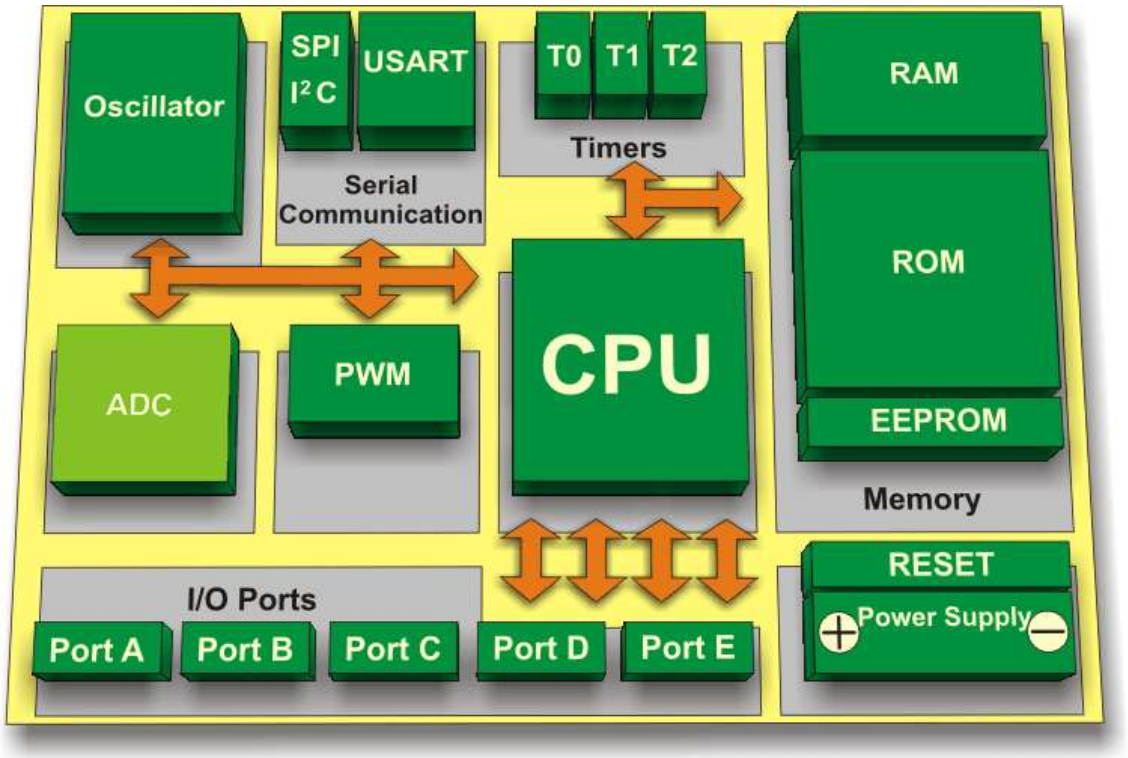
Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

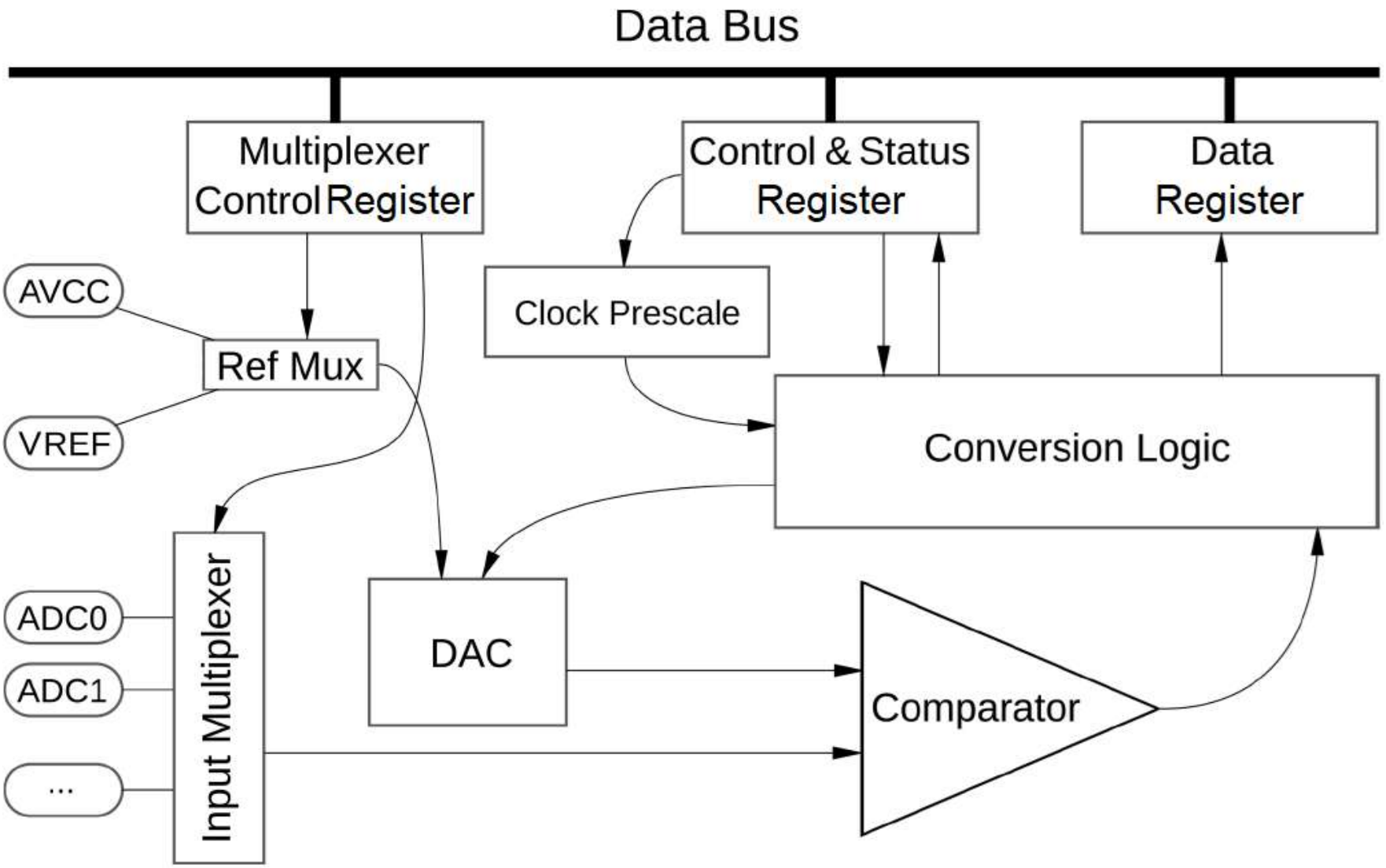
- On completion of this session, you should be able to
  - Explain the need for ADC
  - Describe how ADC converts an analog input into a discrete number
  - Explain the functionality of ADC registers
  - Construct a microcontroller-based system and program it to read an analog input, convert it into a discrete number and then display the result

Analog to Digital Converter

- In physical world, everything is analog
- In computers, everything is digital
- Analog sensors transform physical quantities into analog voltages
- ADC converts analog voltages into discrete numbers

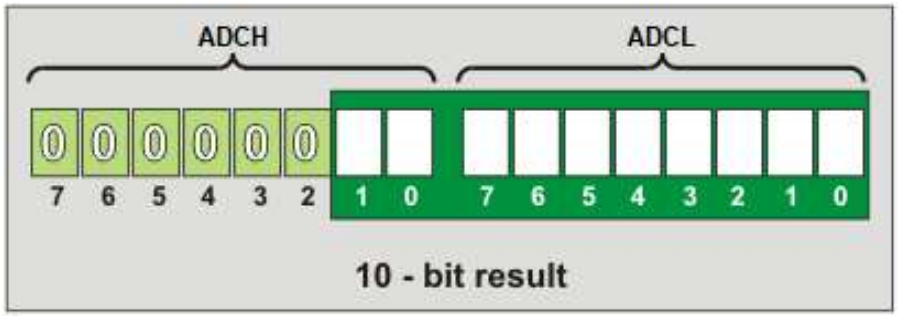


Analog to Digital Conversion



ADC Registers

- ADCL (ADC Low) and ADCH (ADC High) registers
  - Hold conversion result



- ADCW (ADC Word) register
  - 16-bits
  - Provides access to 10-bit result with six leading 0s

ADC Reference Voltage

- ADC converts any analog input voltage between 0 and Vref to a number of discrete voltage intervals between 0 and $2^{\text{resolution}} - 1$
- Resolution is the number of bits used for ADC
- Total number of intervals = $2^{\text{resolution}}$
- Interval size = Vref / $2^{\text{resolution}}$
- analog voltage = number of intervals × interval size

$$\text{Vin} = \text{ADCW} \times \frac{\text{Vref}}{2^{\text{resolution}}}$$

- Interval size is the smallest change in analog input voltage that ADC can distinguish

ADC Registers

- ADMUX (ADc MUltipleXer selection) register



  - REFS1 (REFerence Selection 1) and REFS0 bits set Vref
  - Used to specify Vref and thereby interval size of ADC
  - In Arduino, use `analogReference(type)`

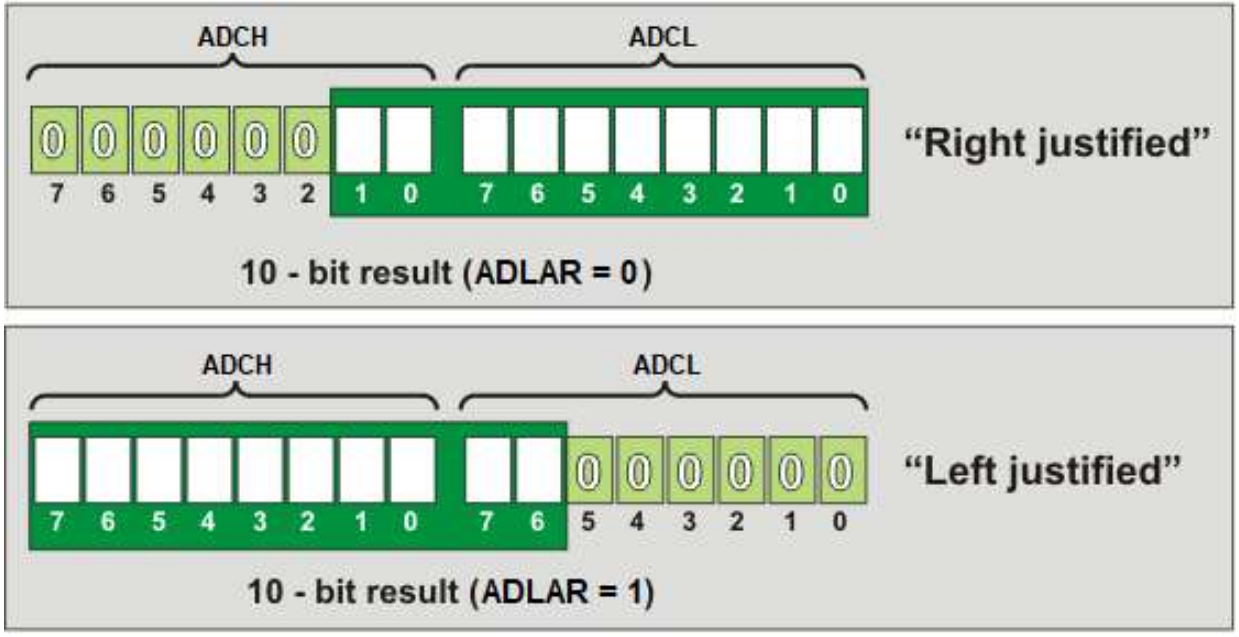
REFS1	REFS0	Vref	type
0	0	Set to V_{AREF} supplied externally	EXTERNAL
0	1	Set to AVcc internally	DEFAULT
1	0	Reserved	
1	1	Set to 1.1V internally	INTERNAL

ADC Registers

- ADMUX register

ADMUX	REFS1	REFS0	ADLAR	–	n/a	MUX2	MUX1	MUX0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

- ADLAR (ADc Left Adjust Result) bit specifies whether the conversion result is left justified



ADC Registers

- ADMUX register



  - MUX2 (MUltipleXer selection 2), MUX1 and MUX0 bits are used to multiplex analog input channels

MUX2	MUX1	MUX0	Channel
0	0	0	ADC0
0	0	1	ADC1
0	1	0	ADC2
0	1	1	ADC3
1	0	0	ADC4
1	0	1	ADC5

ADC Registers

- ADCSRA (ADc Control and Status Register A)



  - ADEN (ADc Enable) bit enables (or disables) ADC
  - Set ADSC (ADc Start Conversion) bit to start ADC
  - ADIF (ADc Interrupt Flag) bit is set when ADC is over and data registers are updated
  - ADIE (ADc Interrupt Enable) bit enables (or disables) ADC complete interrupt

ADC Registers

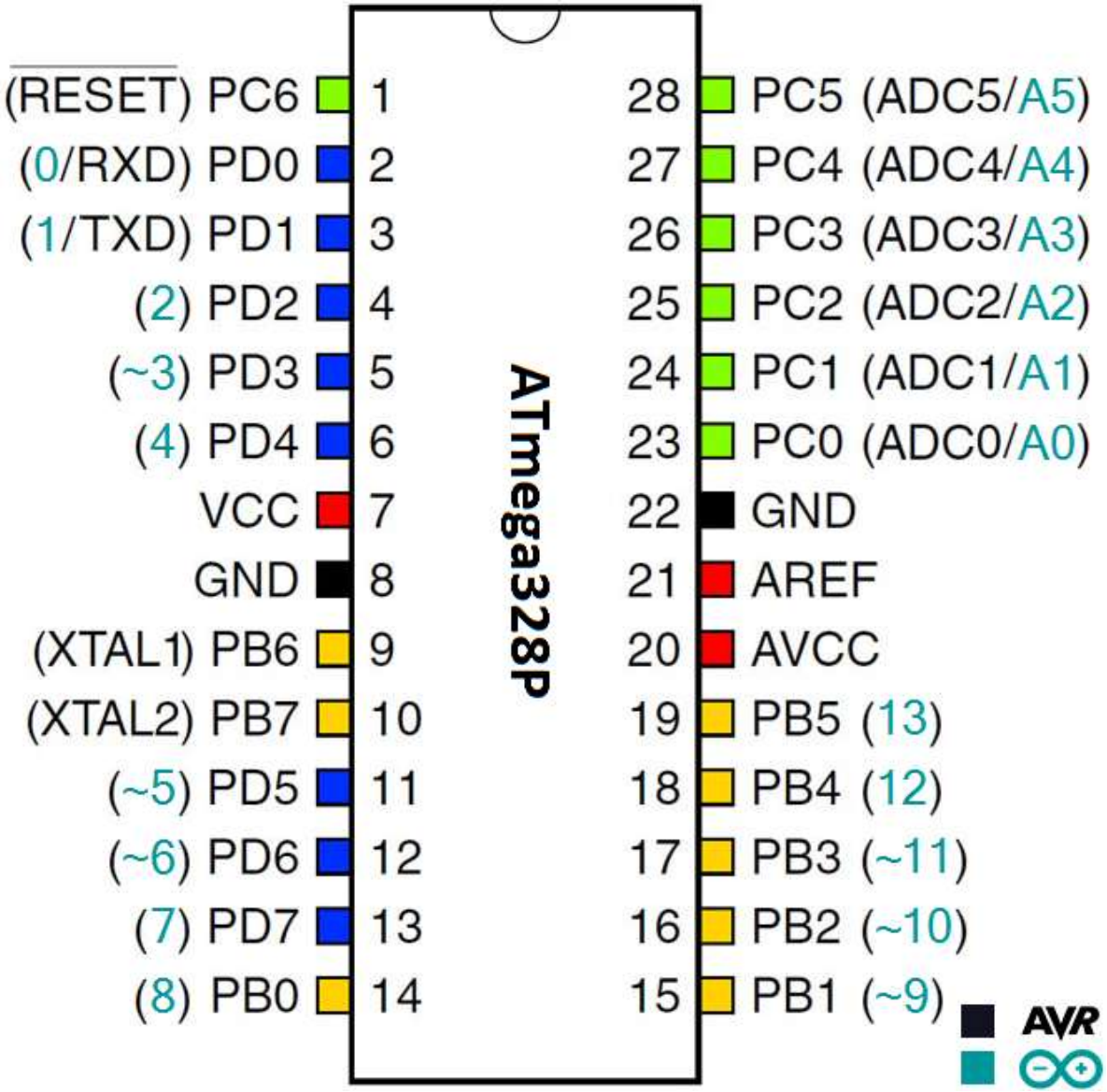
- ADCSRA (ADc Control and Status Register A)
  - ADPS2 (ADc Prescaler Select 2), ADPS1 and ADPS0 bits specify prescale of CPU clock frequency for ADC

ADPS2	ADPS1	ADPS0	F_ADC
0	0	0	Reserved
0	0	1	F_CPU/2
0	1	0	F_CPU/4
0	1	1	F_CPU/8
1	0	0	F_CPU/16
1	0	1	F_CPU/32
1	1	0	F_CPU/64
1	1	1	F_CPU/128

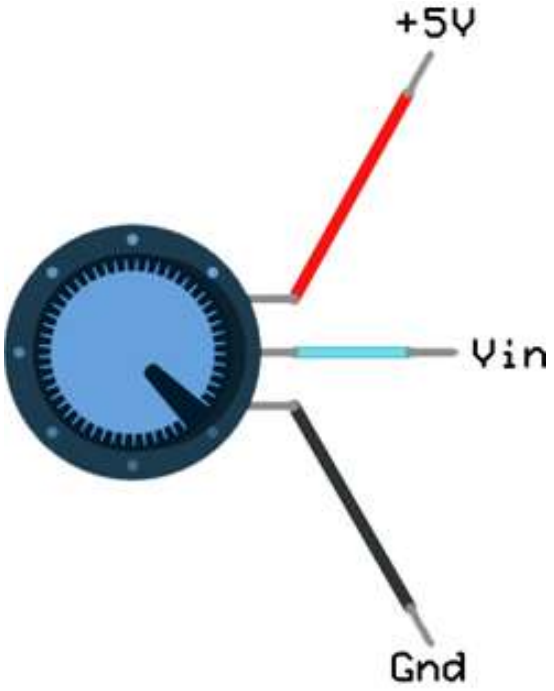
System 8

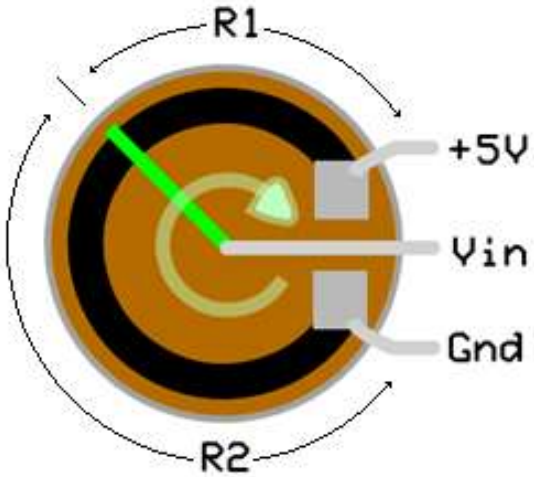
voltMeter - Measures and displays any voltage between 0 and 5V with a set of LEDs

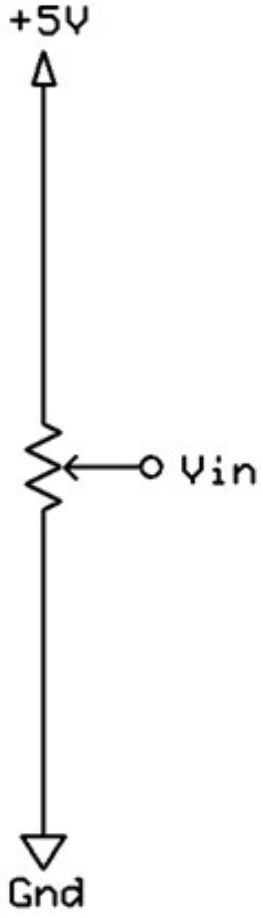
ATmega328P ADC Pinout

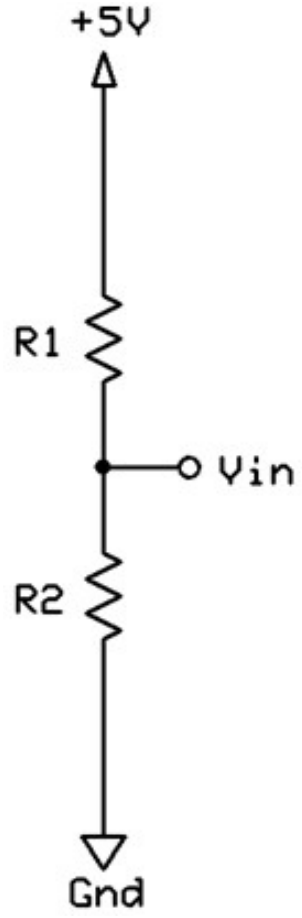


Potentiometer





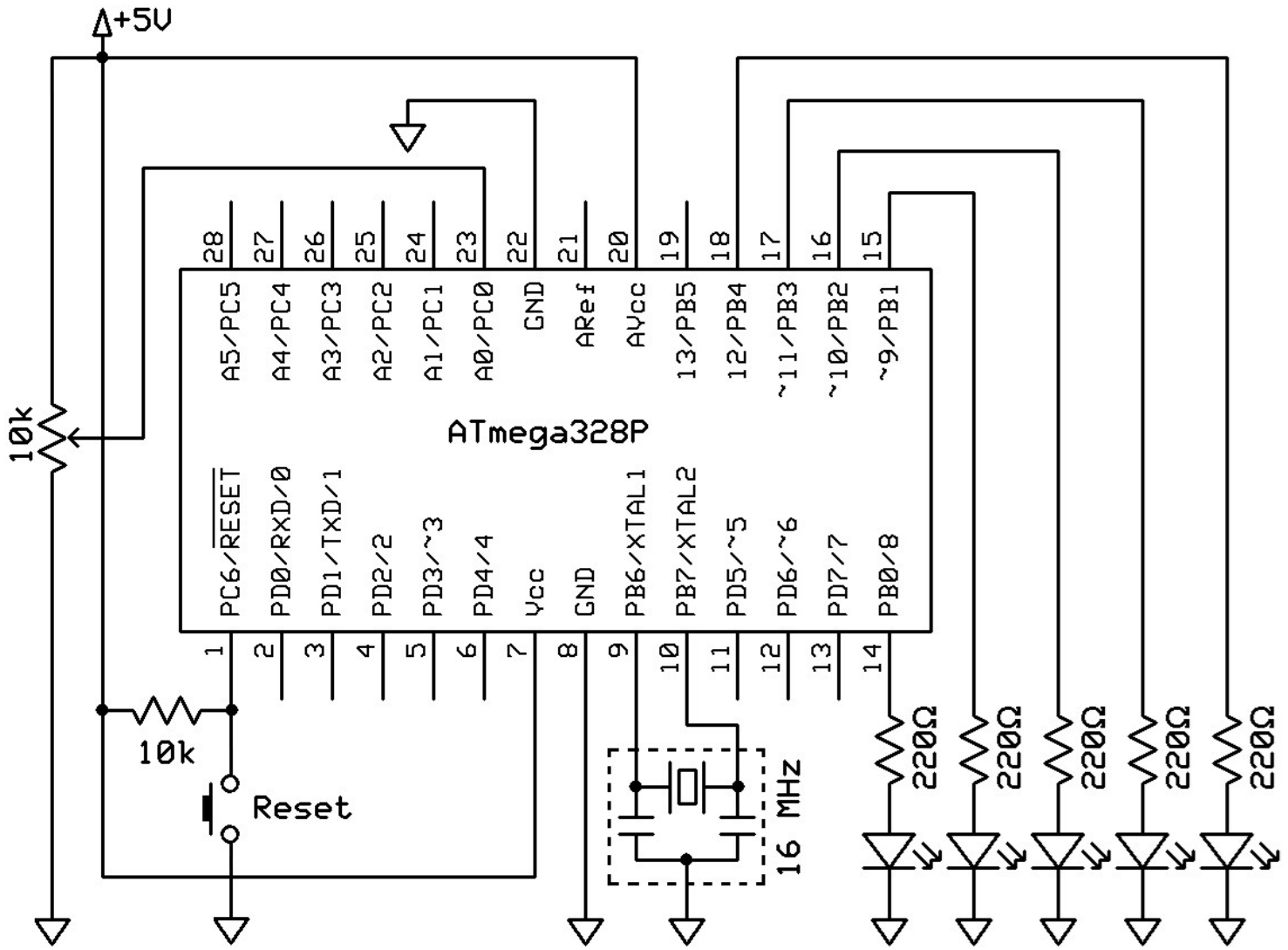




$$Vin = \frac{V \times R2}{R1 + R2}$$

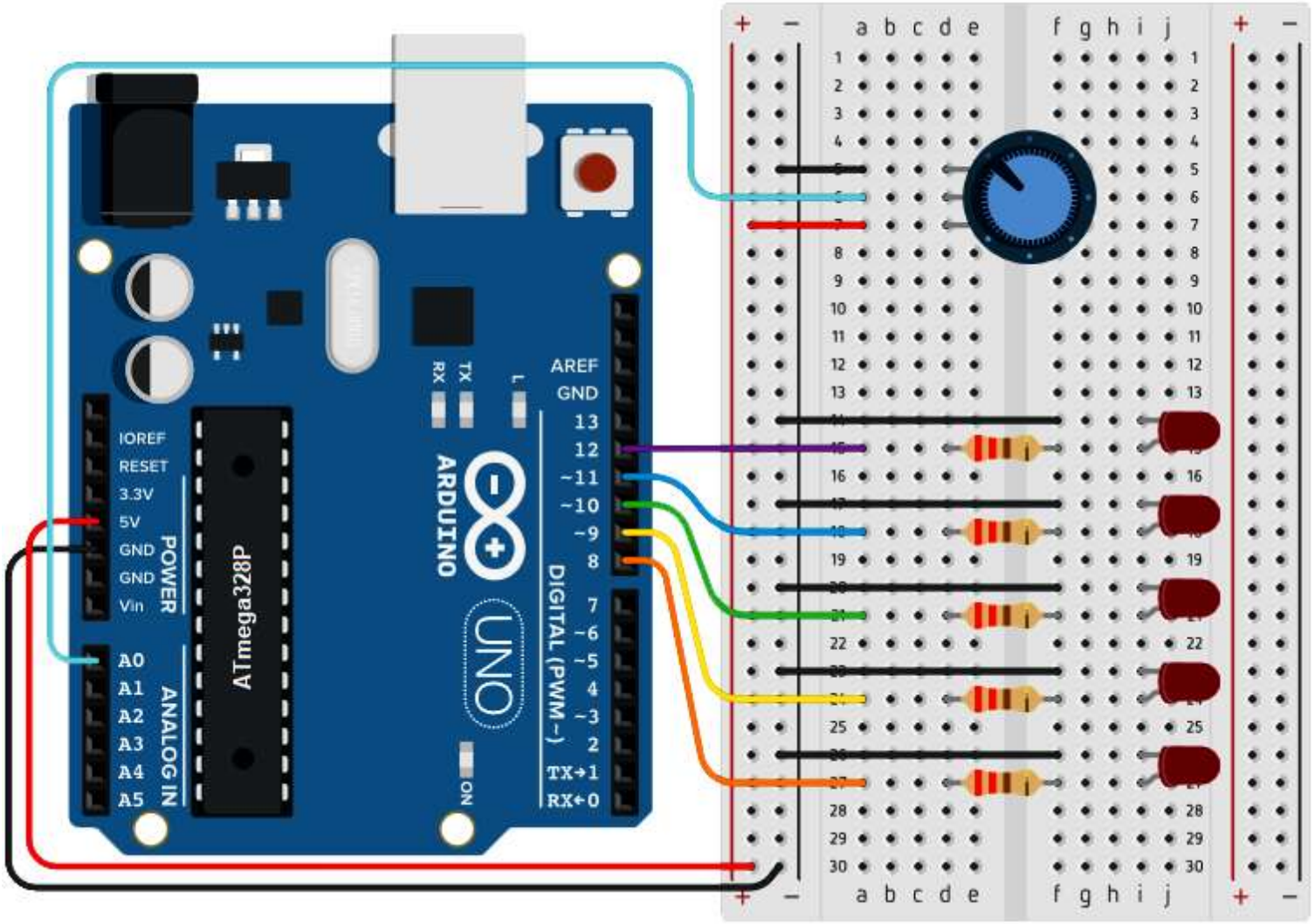
Hardware

- Schematic Diagram



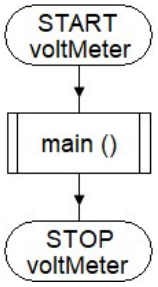
Hardware

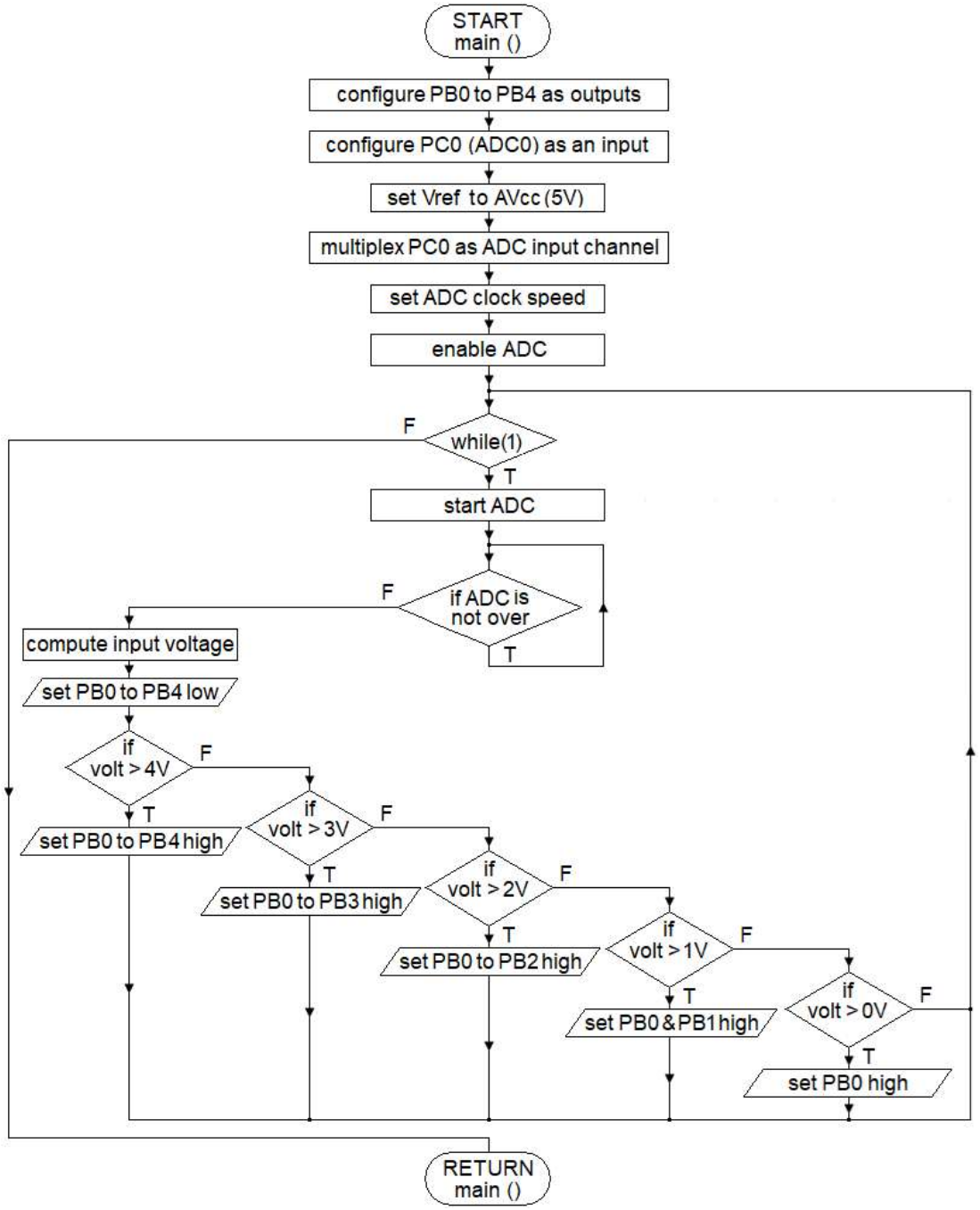
- Wiring Diagram



Firmware

- Algorithm



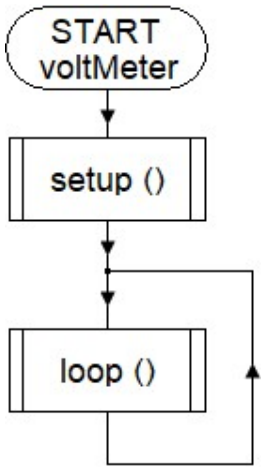


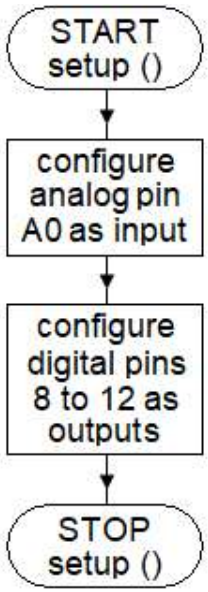
C Program

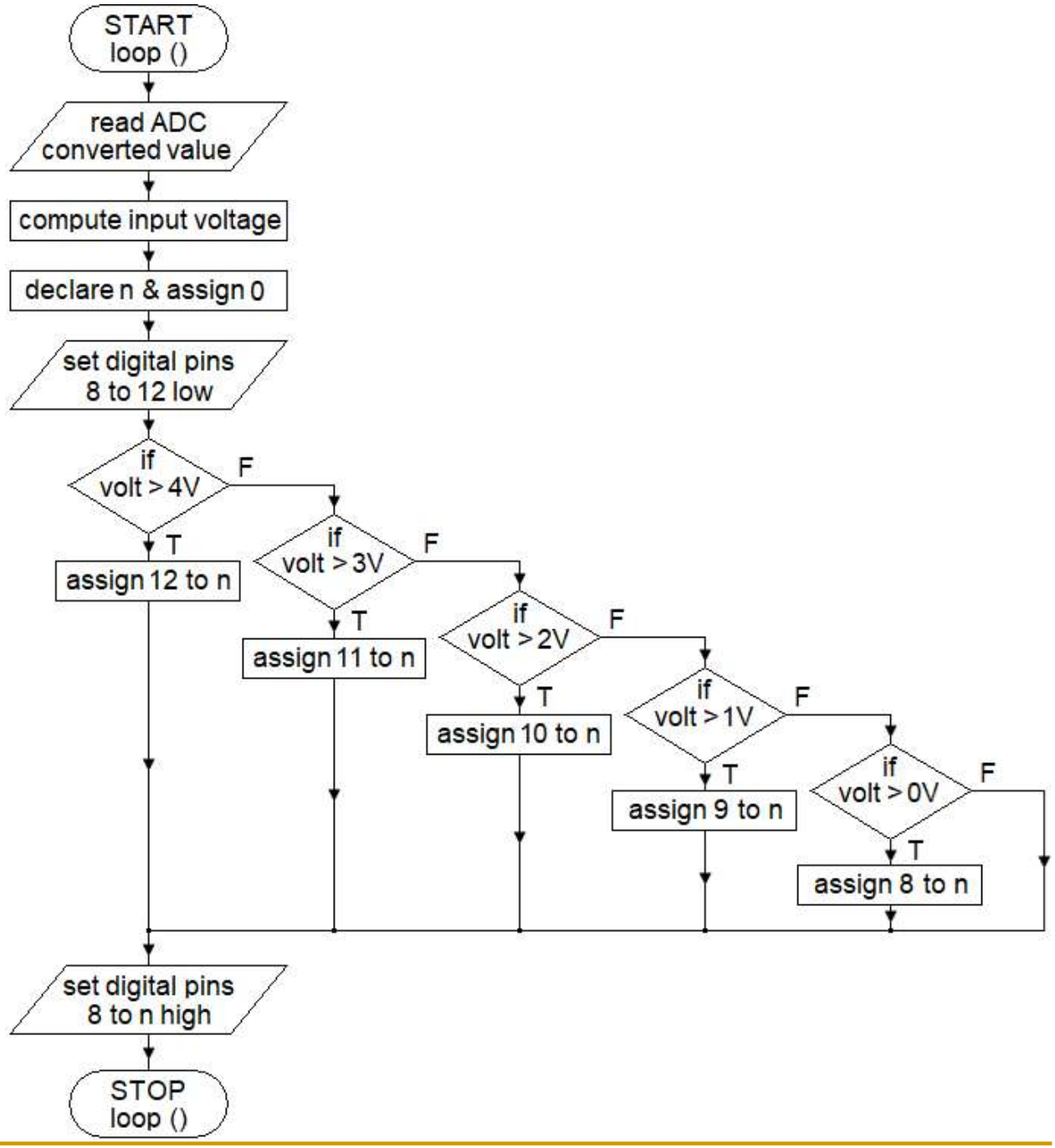
```
1   // voltMeterC.c
2   #include <avr/io.h>                          // include I/O definitions
3   int main(void)
4   {
5       DDRB = DDRB | 0b00011111;                // configure PB0 to PB4 as outputs
6       DDRC = DDRC & 0b11111110;                // configure PC0 (ADC0) as an input
7       ADMUX = (ADMUX & 0b01111111) | 0b01000000; // set Vref to AVcc (5V)
8       ADMUX = ADMUX & 0b11111000;              // multiplex PC0 as analog input channel (ADC0)
9       ADCSRA = ADCSRA | 0b00000111;            // set ADC clock speed to F_CPU/128
10      ADCSRA = ADCSRA | 0b10000000;            // enable ADC
11      while (1)
12      {
13          ADCSRA = ADCSRA | 0b01000000;        // start ADC
14          while(ADCSRA & 0b01000000);          // poll till ADC is over
15          float vIn = ADCW * 5.0 / 1024;       // compute analog input voltage
16          PORTB = PORTB & 0b11100000;          // set PB0 to PB4 low & turn off LEDs
17          if (vIn > 4)
18             PORTB = PORTB | 0b00011111;       // if voltage > 4V, set PB0 to PB4 high & turn on 5 LEDs
19          else if (vIn > 3)
20             PORTB = PORTB | 0b00001111;       // if voltage > 3V, set PB0 to PB3 high & turn on 4 LEDs
21          else if (vIn > 2)
22             PORTB = PORTB | 0b00000111;       // if voltage > 2V, set PB0 to PB2 high & turn on 3 LEDs
23          else if (vIn > 1)
24             PORTB = PORTB | 0b00000011;       // if voltage > 1V, set PB0 to PB1 high & turn on 2 LEDs
25          else if (vIn > 0)
26             PORTB = PORTB | 0b00000001;       // if voltage > 0V, set PB0 & turn on 1 LED
27      }
28      return 0;
29  }
```

Firmware

- Algorithm for Arduino







Firmware

- Source Code (using Arduino Wrapper Functions)

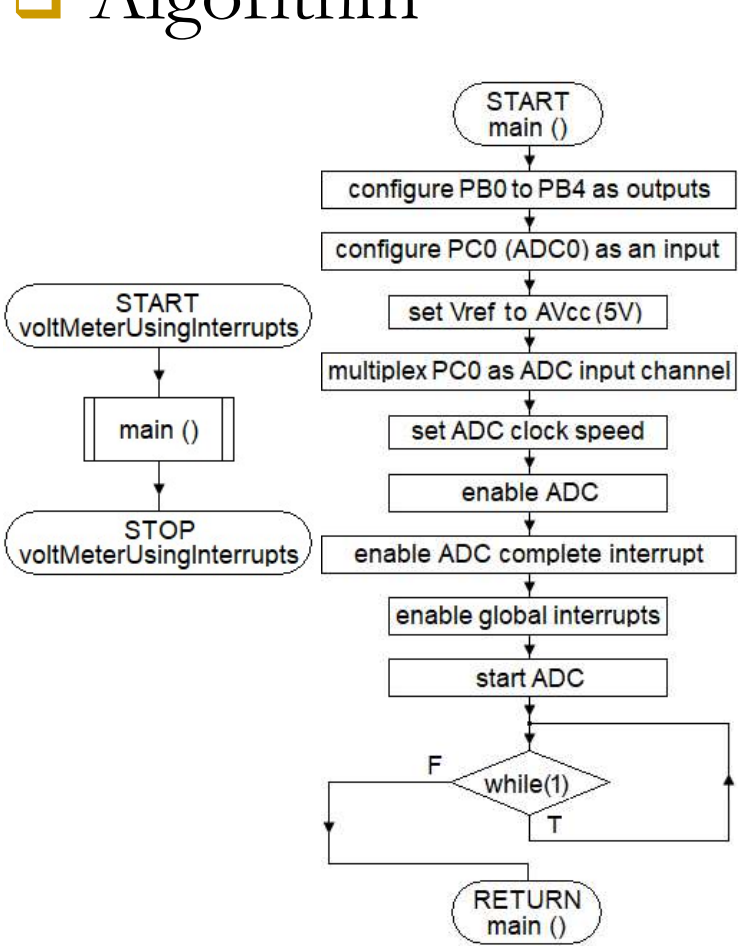
```
1 // voltMeter.ino
2 void setup()
3 {
4   pinMode(A0, INPUT);                  // configure A0 pin as an input (not necessary)
5   for (int pin = 8; pin <= 12; pin++) // configure I/O pins 8 to 12 as outputs
6     pinMode(pin, OUTPUT);
7 }
8 void loop()
9 {
10  int value = analogRead(A0);          // read converted value of analog input voltage
11  float vIn = value * 5.0 / 1024;      // compute analog input voltage
12  for (int pin = 8; pin <= 12; pin++) // set I/O pins 8 to 12 low & turn off LEDs
13    digitalWrite(pin, LOW);
14  int n = 0;                           // declare n and initialize with 0
15  if (vIn > 4)                         // if voltage > 4V, assign 12 to n
16    n = 12;
17  else if (vIn > 3)                    // if voltage > 3V, assign 11 to n
18    n = 11;
19  else if (vIn > 2)                    // if voltage > 2V, assign 10 to n
20    n = 10;
21  else if (vIn > 1)                    // if voltage > 1V, assign 9 to n
22    n = 9;
23  else if (vIn > 0)                    // if voltage > 0V, assign 8 to n
24    n = 8;
25  for (int pin = 8; pin <= n; pin++)  // set I/O pins 8 to n high & turn on LEDs
26    digitalWrite(pin, HIGH);
27 }
```

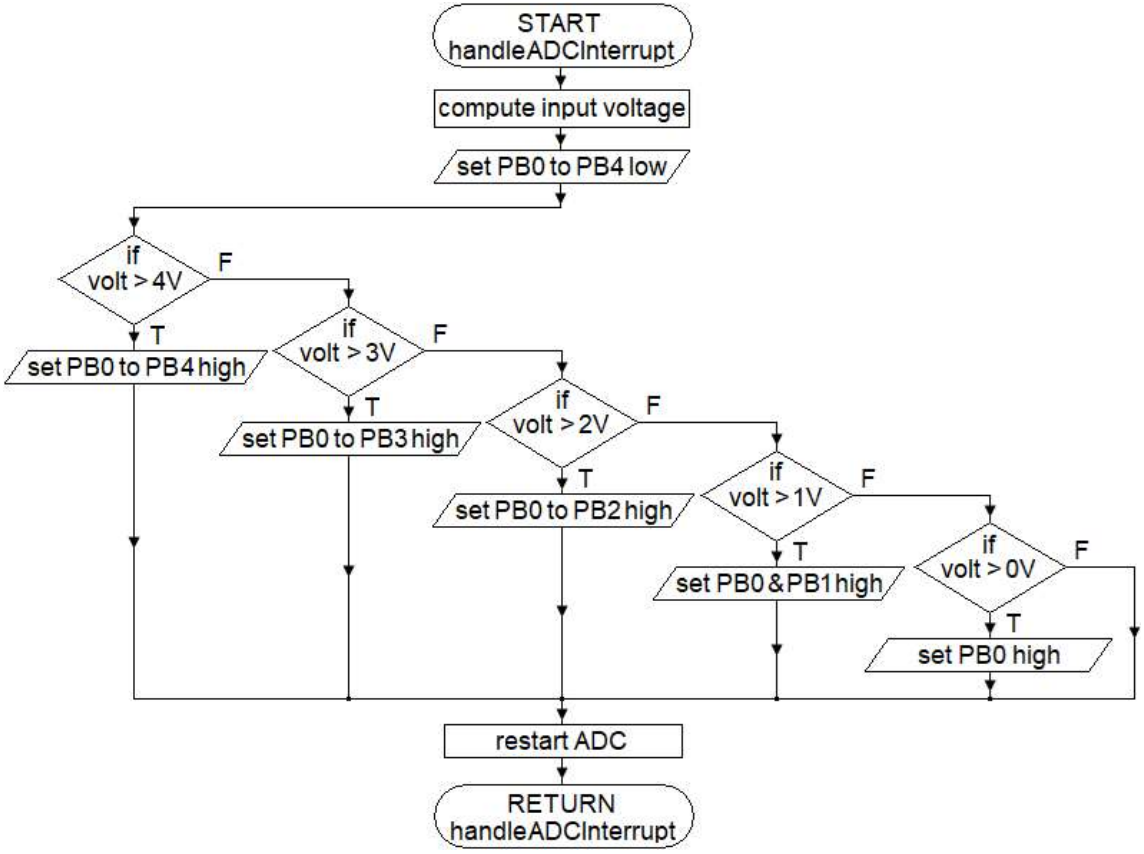
System 9

voltMeterUsingInterrupts - Measures and displays any voltage between 0 and 5V with a set of LEDs using interrupts

Firmware

- Algorithm





C Program

```c
// voltMeterUsingInterruptsC.c
#include <avr/io.h>
#include <avr/interrupt.h>
int main(void)
{
    DDRB = DDRB | 0b00011111;                  // configure PB0 to PB4 as outputs
    DDRC = DDRC & 0b11111110;                  // configure PC0 (ADC0) as an input
    ADMUX = (ADMUX & 0b01111111) | 0b01000000; // set Vref to AVcc (5V)
    ADMUX = ADMUX & 0b11111000;                // multiplex PC0 as analog input channel (ADC0)
    ADCSRA = ADCSRA | 0b00000111;              // set ADC clock speed to F_CPU/128
    ADCSRA = ADCSRA | 0b10000000;              // enable ADC
    ADCSRA = ADCSRA | 0b00001000;              // enable ADC complete interrupt
    sei();                                     // enable global interrupts
    ADCSRA = ADCSRA | 0b01000000;              // start ADC
    while (1)
    {                                          // can be used to perform another task
    }
    return 0;
}
ISR(ADC_vect)
{
   float vIn = ADCW * 5.0 / 1024;              // compute analog input voltage
   PORTB = PORTB & 0b11100000;                 // set PB0 to PB4 low & turn off LEDs
   if (vIn > 4)
       PORTB = PORTB | 0b00011111;             // if voltage > 4V, set set PB0 to PB4 high & turn on 5 LEDs
   else if (vIn > 3)
       PORTB = PORTB | 0b00001111;             // if voltage > 3V, set set PB0 to PB3 high & turn on 4 LEDs
   else if (vIn > 2)
       PORTB = PORTB | 0b00000111;             // if voltage > 2V, set set PB0 to PB2 high & turn on 3 LEDs
   else if (vIn > 1)
       PORTB = PORTB | 0b00000011;             // if voltage > 1V, set set PB0 & PB1 high & turn on 2 LEDs
   else if (vIn > 0)
       PORTB = PORTB | 0b00000001;             // if voltage > 0V, set set PB0 high & turn on 1 LED
   ADCSRA = ADCSRA | 0b01000000;               // restart ADC
}
```

Using Hexadecimal Constants

```
1   // voltMeterUsingInterruptsC.c
2   #include <avr/io.h>
3   #include <avr/interrupt.h>
4   int main(void)
5   {
6       DDRB = DDRB | 0x1F;                  // configure PB0 to PB4 as outputs
7       DDRC = DDRC & 0xFE;                  // configure PC0 (ADC0) as an input
8       ADMUX = (ADMUX & 0x7F) | 0x40;       // set Vref to AVcc (5V)
9       ADMUX = ADMUX & 0xF8;                // multiplex PC0 as analog input channel (ADC0)
10      ADCSRA = ADCSRA | 0x07;              // set ADC clock speed to F_CPU/128
11      ADCSRA = ADCSRA | 0x80;              // enable ADC
12      ADCSRA = ADCSRA | 0x08;              // enable ADC complete interrupt
13      sei();                               // enable global interrupts
14      ADCSRA = ADCSRA | 0x40;              // start ADC
15      while (1)
16      {                                    // can be used to perform another task
17      }
18      return 0;
19  }
20  ISR(ADC_vect)
21  {
22     float vIn = ADCW * 5.0 / 1024;        // compute analog input voltage
23     PORTB = PORTB & 0xE0;                 // set PB0 to PB4 low & turn off LEDs
24     if (vIn > 4)
25         PORTB = PORTB | 0x1F;             // if voltage > 4V, set set PB0 to PB4 high & turn on 5 LEDs
26     else if (vIn > 3)
27         PORTB = PORTB | 0x0F;             // if voltage > 3V, set set PB0 to PB3 high & turn on 4 LEDs
28     else if (vIn > 2)
29         PORTB = PORTB | 0x07;             // if voltage > 2V, set set PB0 to PB2 high & turn on 3 LEDs
30     else if (vIn > 1)
31         PORTB = PORTB | 0x03;             // if voltage > 1V, set set PB0 & PB1 high & turn on 2 LEDs
32     else if (vIn > 0)
33         PORTB = PORTB | 0x01;             // if voltage > 0V, set set PB0 high & turn on 1 LED
34     ADCSRA = ADCSRA | 0x40;               // restart ADC
35  }
```

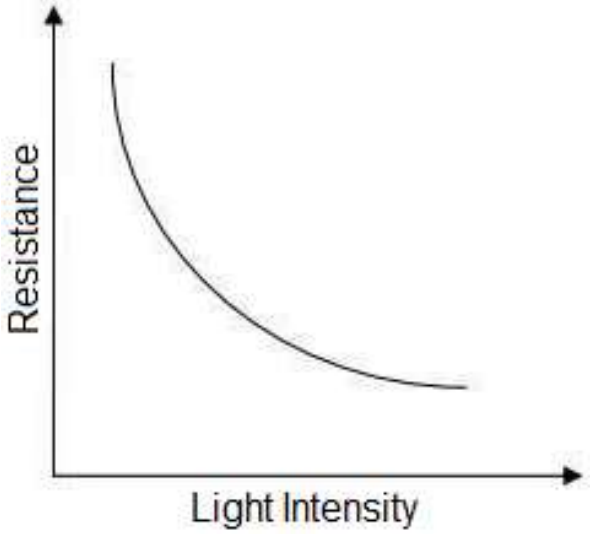
Using Labeled Constants

```
1   // voltMeterUsingInterruptsC.c
2   #include <avr/io.h>
3   #include <avr/interrupt.h>
4   int main(void)
5   {
6       DDRB = DDRB | (1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
7       DDRC = DDRC & ~(1<<PORTC0);
8       ADMUX = (ADMUX & ~(1<<REFS1)) | (1<<REFS0);
9       ADMUX = ADMUX & ~((1<<MUX2) | (1<<MUX1) | (1<<MUX0));
10      ADCSRA = ADCSRA | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);
11      ADCSRA = ADCSRA | (1<<ADEN);
12      ADCSRA = ADCSRA | (1<<ADIE);
13      sei();
14      ADCSRA = ADCSRA | (1<<ADSC);
15      while (1)
16      {
17      }
18      return 0;
19  }
20  ISR(ADC_vect)
21  {
22     float vIn = ADCW * 5.0 / 1024;
23     PORTB = PORTB & ~((1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0));
24     if (vIn > 4)
25         PORTB = PORTB | (1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
26     else if (vIn > 3)
27         PORTB = PORTB | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
28     else if (vIn > 2)
29         PORTB = PORTB | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
30     else if (vIn > 1)
31         PORTB = PORTB | (1<<PORTB1) | (1<<PORTB0);
32     else if (vIn > 0)
33         PORTB = PORTB | (1<<PORTB0);
34     ADCSRA = ADCSRA | (1<<ADSC);
35  }
```

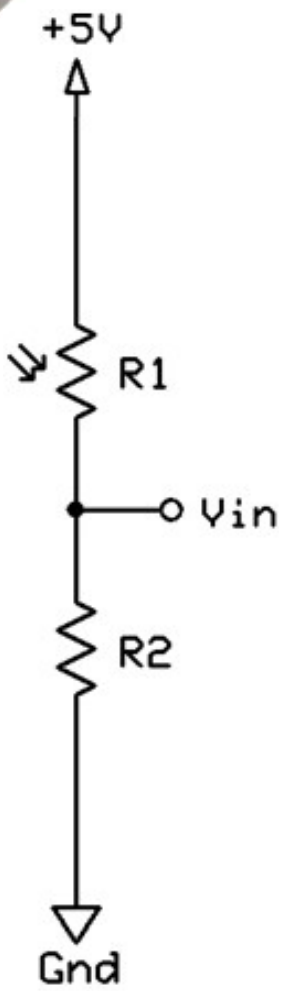
Light Sensor

- Light Dependent Resistor (LDR)
  - Changes resistance on the intensity of light falls upon
  - Typically used in a voltage divider to get converted the light intensity into a voltage





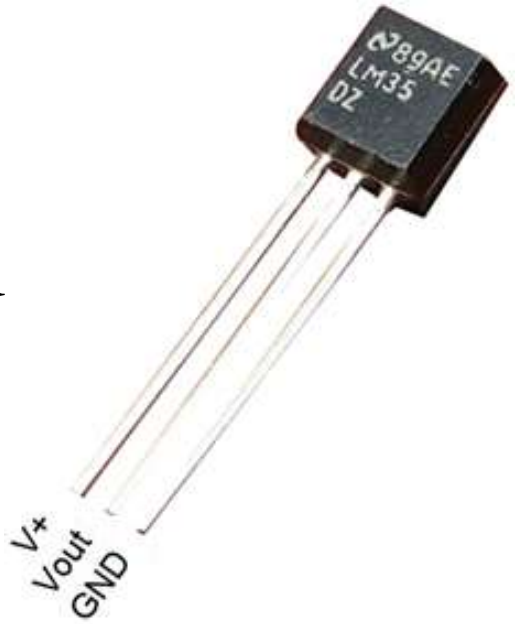
$$Vin = \frac{V \times R2}{R1 + R2}$$



Temperature Sensor

- LM35 from Texas Instruments
  - Provides a voltage linearly proportional to the temperature sensed
  - Sensitive to temperatures from −55°C to +150°C

$$Temperature = Vout \times 100$$



Questions